DATABRICKS BOOTCAMP · DAY 1 HOMEWORK

Lakebase Support Ticketing System

A full-stack internal IT support portal built on Databricks Lakebase — managed PostgreSQL OLTP — and deployed as a Databricks App. Covers schema design, an OAuth-secured data layer, a corporate-grade Gradio interface, and end-to-end smoke testing against live data.

AUTHOR	Jayanth Dolai — Senior Software Engineer, UnitedHealth Group / Optum
LIVE APP	https://support-ticketing-7474654640109575.aws.databricksapps.com
SOURCE	github.com/demonjd2026-afk/lakebase-support-ticketing
PLATFORM	Databricks Free Edition (AWS) · Lakebase Postgres · Databricks Apps
FRONTEND	Gradio (Python) — three-tab corporate ticketing portal
DATA LAYER	psycopg2 over the standard Postgres wire protocol, TLS enforced
AUTH MODEL	Service-principal OAuth (client_credentials) → short-lived Lakebase JWT

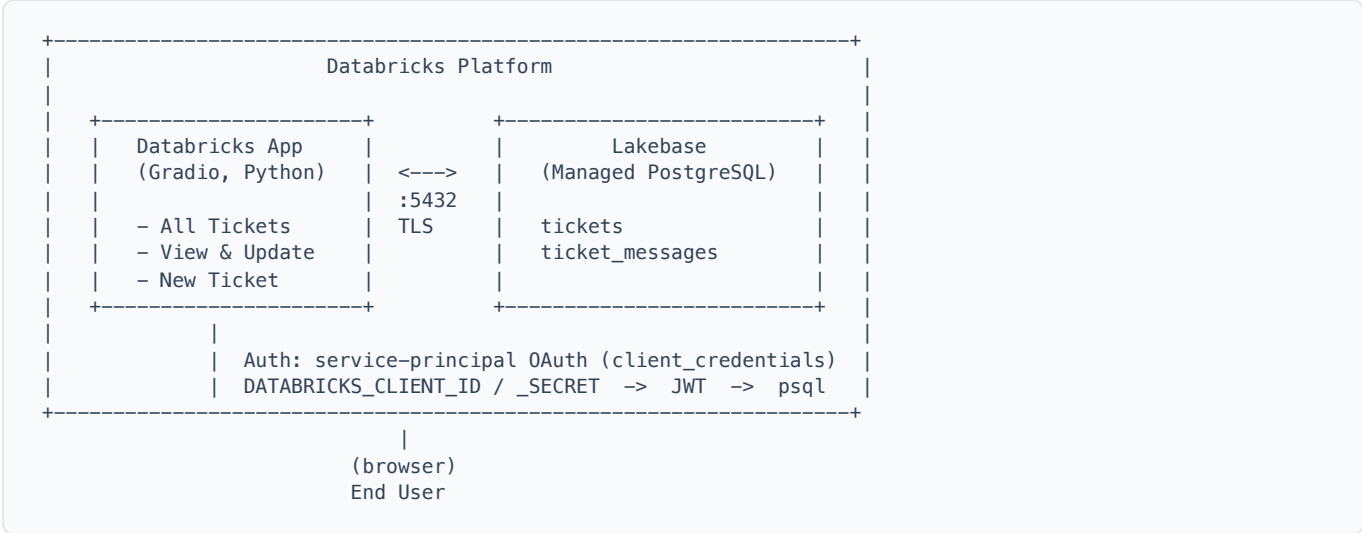
Project highlights

- Two related Postgres tables (`tickets`, `ticket_messages`) with a foreign key and cascading delete
- Seeded with 4 tickets across all 3 statuses and 11 threaded messages; grown to 5 tickets / 13 messages through live app use
- Full CRUD — view, filter, create, add message, update status, delete behind a confirmation guard
- Bonus features: priority & category fields, live dashboard statistics, input validation, custom-styled UI
- No credential ever reaches source control — the app mints a fresh OAuth JWT per connection using its own service-principal identity

Contents

1 Architecture Overview

Three tiers in a single process: a Gradio frontend running as a Databricks App, a thin Python data-access module, and Lakebase — Databricks' managed PostgreSQL OLTP engine — as the system of record.



app/app.py	Gradio UI, event handlers, and ~370 lines of custom CSS. Contains no SQL.
app/db.py	OAuth token exchange, connection management, and every query in the system.
Lakebase	The system of record: two tables, three indexes, one foreign key with cascade.

Why Lakebase instead of a Delta table?

DIMENSION	DELTA LAKE (ANALYTICS)	LAKEBASE (OLTP)
Designed for	Batch reads, BI, ML	Transactional reads / writes
Latency	Seconds to minutes	Milliseconds
Consistency	Eventually consistent	ACID, row-level
Write pattern	Append / bulk merge	Insert / Update / Delete
Concurrency	File-level commits	Row-level locking
Best fit	Dashboards, pipelines	Apps, APIs, operational data

A ticketing app is a stream of single-row inserts and updates from concurrent users. That is precisely the workload Postgres was designed for, and precisely the workload a Delta table handles badly.

PHASE 1

2 Lakebase Schema & Seed Data

Provisioned a Lakebase project, defined two related Postgres tables with supporting indexes, and seeded realistic sample data — all executed directly in the Lakebase SQL Editor.

Project	`support-ticketing`
Branch	`production`
Database	`databricks_postgres`
Region	`us-east-2` (AWS)
Auth type	OAuth — Databricks identities & service principals

Schema DDL

```
CREATE TABLE tickets (
    ticket_id   SERIAL       PRIMARY KEY,
    title       TEXT         NOT NULL,
    status      TEXT         NOT NULL DEFAULT 'open',    -- open | in_progress | resolved
    priority    TEXT         NOT NULL DEFAULT 'medium',  -- low | medium | high
    category    TEXT,                                    -- infra | access | data | billing
    created_by  TEXT         NOT NULL,
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE ticket_messages (
    message_id   SERIAL       PRIMARY KEY,
    ticket_id    INT          NOT NULL REFERENCES tickets(ticket_id) ON DELETE CASCADE,
    message_text TEXT         NOT NULL,
    author       TEXT         NOT NULL,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_ticket_id ON ticket_messages(ticket_id);
CREATE INDEX idx_tickets_status     ON tickets(status);
CREATE INDEX idx_tickets_priority   ON tickets(priority);
```

`ON DELETE CASCADE` is what reduces "delete a ticket and its whole thread" to a single statement — Postgres removes the messages as part of the same transaction.

Seed data & verification

`sql/02_seed_data.sql` inserts **4 tickets covering all three statuses** with **11 threaded messages** — comfortably above the assignment minimum of 3 tickets, 2 statuses and 2 messages each. The state below was captured after live testing through the deployed app, which added a fifth ticket and two further messages.

```
SELECT t.ticket_id, t.title, t.status, t.priority,
       COUNT(m.message_id) AS message_count
FROM tickets t
LEFT JOIN ticket_messages m ON t.ticket_id = m.ticket_id
```

```
GROUP BY t.ticket_id, t.title, t.status, t.priority
ORDER BY t.ticket_id;
```

ID	TITLE	STATUS	PRIORITY	MSGS
1	Databricks cluster auto-terminates during pipeline run	in_progress	high	4
2	Unable to access Unity Catalog schema after permission update	in_progress	high	3
3	Delta table OPTIMIZE job taking longer than expected	resolved	medium	3
4	Lakebase connection string not recognized in Databricks App	open	medium	2
5	Spark job failing with OOM error on large dataset	in_progress	high	1

Result: 5 tickets across 3 distinct statuses and 13 messages at capture time, returned by Lakebase in 405 ms (see Fig. 8).

PHASE 2

3 Data Access Layer (db.py)

A single module isolating all SQL. The UI never writes a query — every read and write goes through a helper function that opens a connection, works inside `try/finally`, commits or rolls back, and closes.

Connection strategy

Lakebase rejects personal access tokens on OAuth-enabled connections — it requires a **JWT**. Inside Databricks Apps the platform injects `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET` for the app's own auto-generated service principal. `db.py` exchanges those for a short-lived token via the OIDC `client_credentials` grant, then uses that token as the Postgres password.

```
def get_oauth_token():
    client_id     = os.environ["DATABRICKS_CLIENT_ID"]
    client_secret = os.environ["DATABRICKS_CLIENT_SECRET"]
    host          = os.environ.get("DATABRICKS_HOST", ...)

    r = requests.post(f"{host}/oidc/v1/token", data={
        "grant_type":    "client_credentials",
        "client_id":     client_id,
        "client_secret": client_secret,
        "scope":         "all-apis",
    }, timeout=30)

    if r.status_code != 200:
        raise RuntimeError(f"Token fetch failed: {r.status_code} {r.text}")
    return r.json()["access_token"]


def get_conn():
    token     = get_oauth_token()
    client_id = os.environ["DATABRICKS_CLIENT_ID"]
    user = urllib.parse.quote(client_id, safe="")   # URL-encode both halves
    pw   = urllib.parse.quote(token,     safe="")
    conn_str = (f"postgresql://{user}:{pw}"
                f"@{LAKEBASE_HOST}/{LAKEBASE_DB}?sslmode=require")
    return psycopg2.connect(conn_str, cursor_factory=RealDictCursor)
```

A fresh token is minted per connection, so no long-lived credential is ever cached in the process or written to disk. `sslmode=require` forces TLS on every connection.

Functions exposed

FUNCTION	PURPOSE
`get_oauth_token()`	Exchange service-principal credentials for a Lakebase JWT
`get_conn()`	Open a TLS psycopg2 connection with `RealDictCursor`
`get_all_tickets(status_filter)`	List tickets with message counts; optional status filter

FUNCTION	PURPOSE
`get_ticket_by_id(id)`	Single ticket lookup, returned as a dict
`get_messages(ticket_id)`	Full message thread, oldest first
`get_stats()`	Counts by status plus total, for the sidebar dashboard
`create_ticket(title, created_by, priority, category)`	Insert and return the new `ticket_id`
`add_message(ticket_id, text, author)`	Insert a threaded message
`update_status(ticket_id, new_status)`	Update status, validated against an allow-list
`delete_ticket(ticket_id)`	Delete, cascading to the ticket's messages

Message counts are computed inside the ticket query as a correlated sub-select, so the All Tickets tab renders from a single round trip rather than N+1 queries.

PHASE 3

4 Application UI (app.py, Gradio)

A three-tab Gradio interface styled to resemble a corporate helpdesk product — top navigation bar, sticky sidebar dashboard, and card-based content tabs.

Tab structure

- **All Tickets** — filterable, sortable table of every ticket with status, priority, category, requester, timestamp and message count columns
- **View & Update** — load a ticket by ID to get a formatted detail card, the full message thread, a status-update control, an add-message form, and a guarded delete action
- **New Ticket** — validated creation form (title, priority, category, requester)
- **Sidebar** — sticky live counts (Total / Open / In Progress / Resolved) that refresh after every write operation, because each mutation handler returns fresh stats HTML alongside its own output

Design system

A light, neutral corporate palette (slate and indigo) modelled on Zendesk / Freshdesk / Jira Service Desk: white cards on a soft slate background, colour-coded status and priority chips, the Inter typeface, and consistent 42–46 px control heights. Gradio's default dark surfaces are overridden with scoped CSS so the app reads as a product rather than a demo.

Validation & guards

Handlers validate before touching the database and return inline feedback rather than raising — empty titles, missing requester names, empty messages and unloaded tickets are all caught in the UI layer, with `db.py` re-validating independently.

```
def on_delete_ticket(tid, confirmed):
    if not tid:       return "⚠️ No ticket loaded", ...
    if not confirmed: return "⚠️ Tick the confirmation box first", ...
    delete_ticket(int(tid))
    return f"✅ Ticket #{int(tid)} deleted", load_ticket_table(), load_stats_html()
```

PHASE 4

5 Deployment on Databricks Apps

Connected the GitHub repository directly to a Databricks App, granted its service principal access to the Lakebase branch, and verified connectivity end to end.

App name	`support-ticketing`
Source	GitHub — `github.com/demonjd2026-afk/lakebase-support-ticketing`, branch `main`, path `app/`
Entrypoint	`python app.py` (declared in `app.yml`)
Port	`DATABRICKS_APP_PORT`, defaulting to `8000`
Runtime auth	Injected `DATABRICKS_CLIENT_ID` / `DATABRICKS_CLIENT_SECRET`
App URL	https://support-ticketing-7474654640109575.aws.databricksapps.com

app.yml

```
command: ["python", "app.py"]

env:
  - name: LAKEBASE_CONN
    valueFrom: LAKEBASE_CONN
```

> **Note on `LAKEBASE_CONN`:** this secret binding is a remnant of the first, PAT-based connection attempt. The deployed code no longer reads it — authentication runs entirely through the injected service-principal credentials described in Phase 2. The binding is harmless and can be removed from `app.yml` without affecting the app.

Access control

The App's auto-generated service principal was granted the `databricks_superuser` role on the Lakebase branch via **Roles & Databases**, so it reads and writes both tables under its own OAuth identity. No personal access token is stored anywhere in the deployment.

Smoke test results

- ✓ Existing tickets load correctly from Lakebase on page open
- ✓ New ticket created via the form persists and appears in the list
- ✓ Message added to a ticket persists and appears in the thread
- ✓ Ticket status updated, reflected immediately, and survives a page refresh
- ✓ Status filter narrows the ticket list correctly
- ✓ Delete requires the confirmation checkbox before removing a ticket
- ✓ Records verified independently in the Lakebase SQL Editor

6 Screenshots — Live Application Walkthrough

Eight captures walking a ticket's full lifecycle — browse, filter, read, reply, re-status, create, delete — and verifying the result in Lakebase itself.

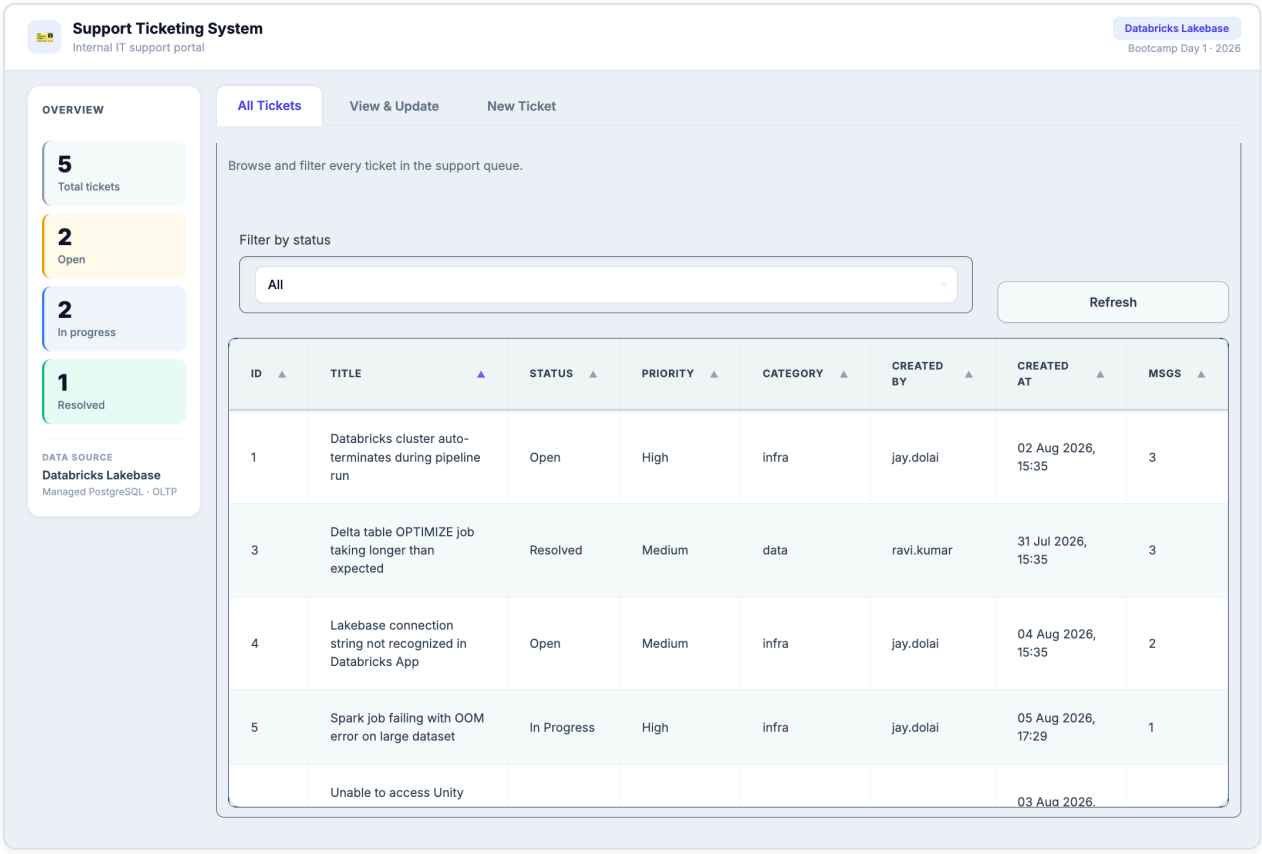


Fig. 1 — All Tickets tab. The landing view. A sticky sidebar reports Total / Open / In Progress / Resolved counts read live from Lakebase, alongside the full ticket queue with status, priority, category, requester, creation time and message count per row.

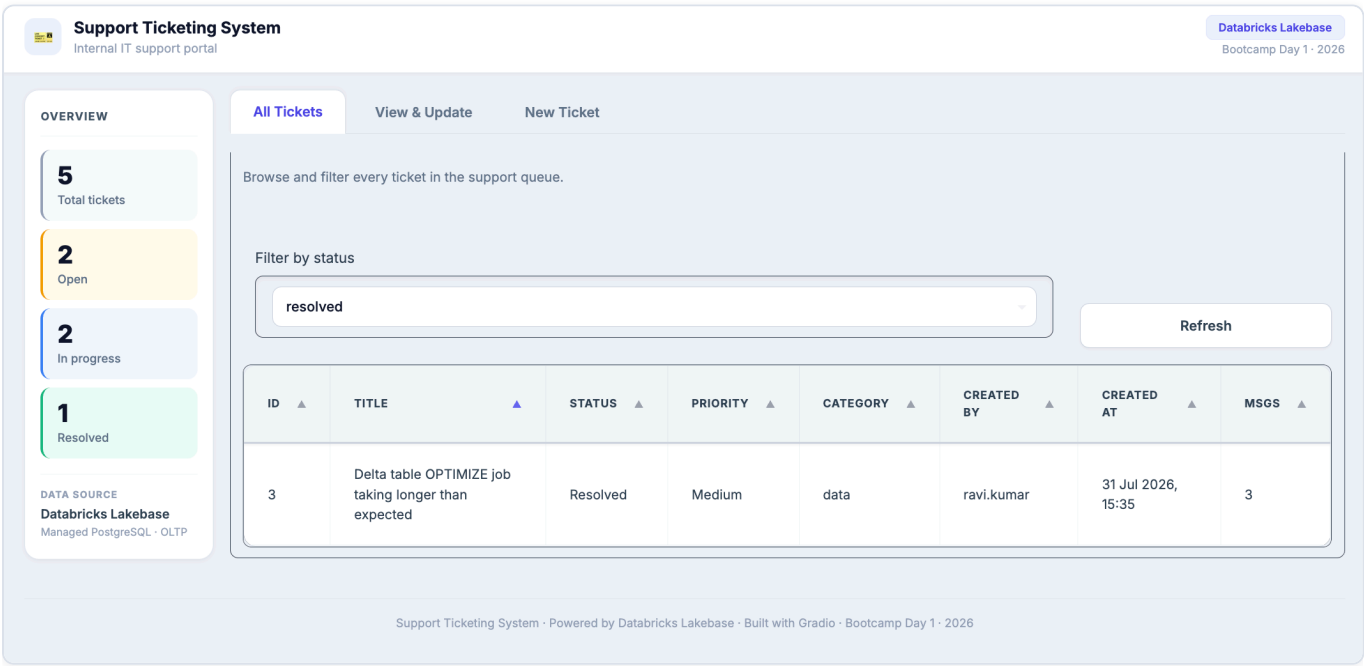


Fig. 2 — Filter by status (bonus). Selecting `resolved` re-queries Lakebase with a `WHERE t.status = %s` predicate and narrows the table to matching tickets only. The sidebar keeps showing overall totals, so filtering is a view concern rather than a data one.

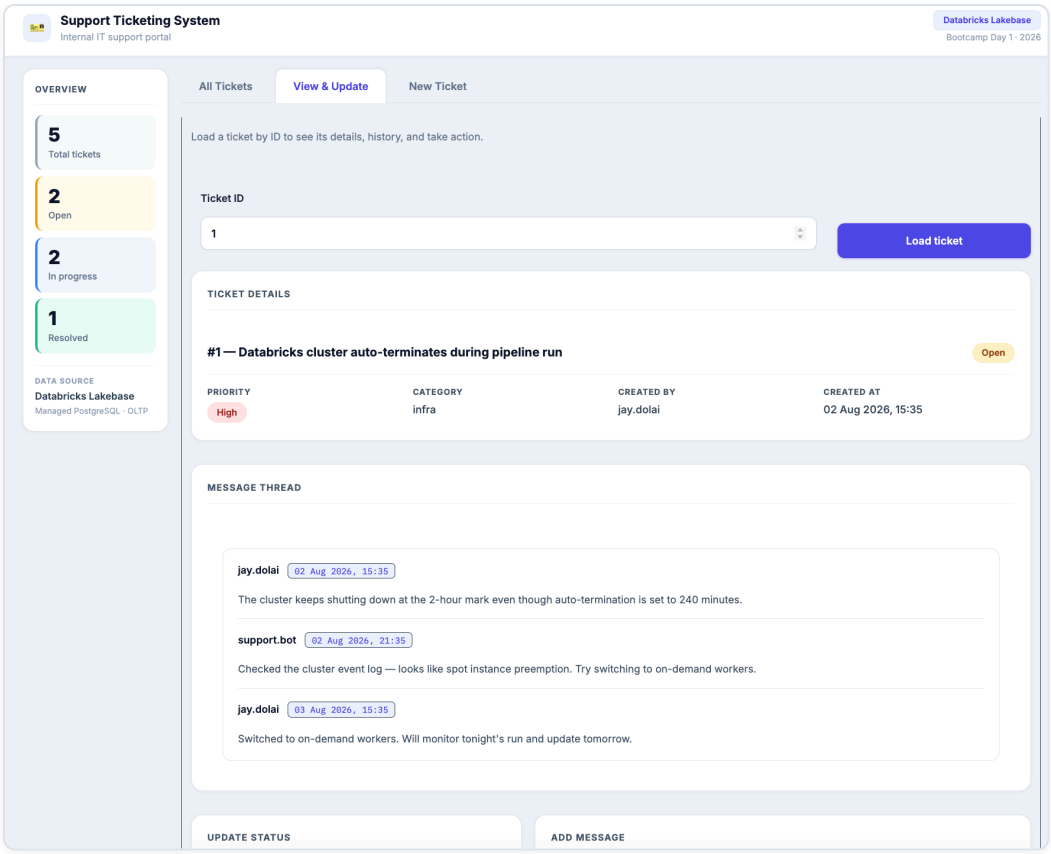


Fig. 3 — View & Update tab. Loading a ticket by ID renders a detail card with colour-coded status and priority chips, followed by the full message thread in chronological order — author and timestamp on every entry.

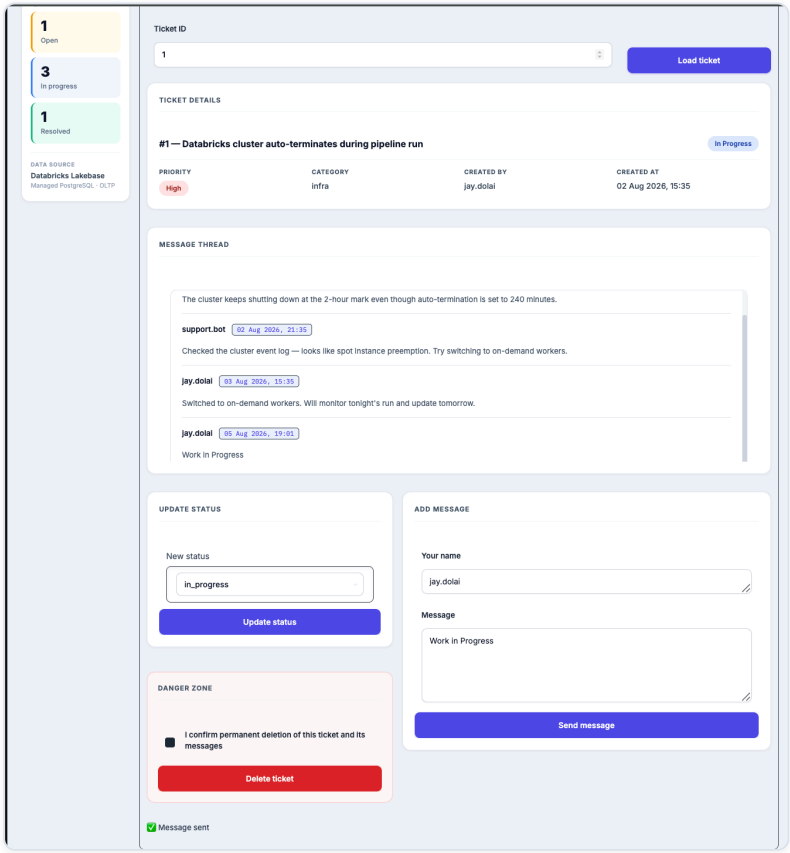


Fig. 4 — Add message. A reply is inserted into `ticket_messages` and the thread re-renders immediately, with inline confirmation feedback beneath the action panels.

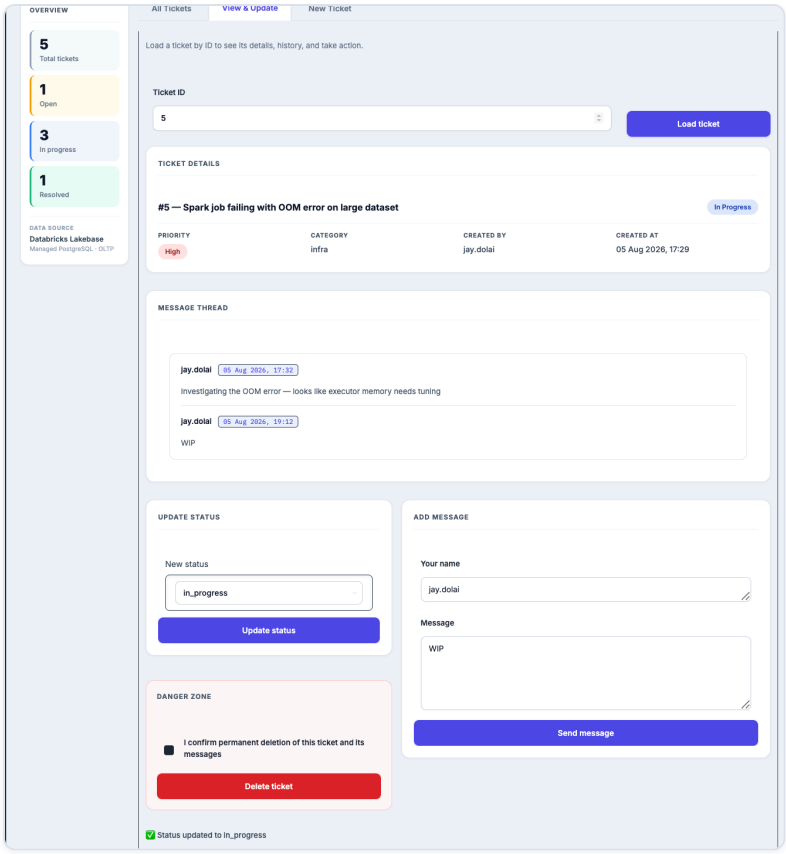


Fig. 5 — Update status. Ticket #5 moved to `in_progress`. The write goes through to Lakebase, the detail card's chip updates, and the sidebar's In Progress counter increments. The change survives a full page refresh — it is in Postgres, not in session state.

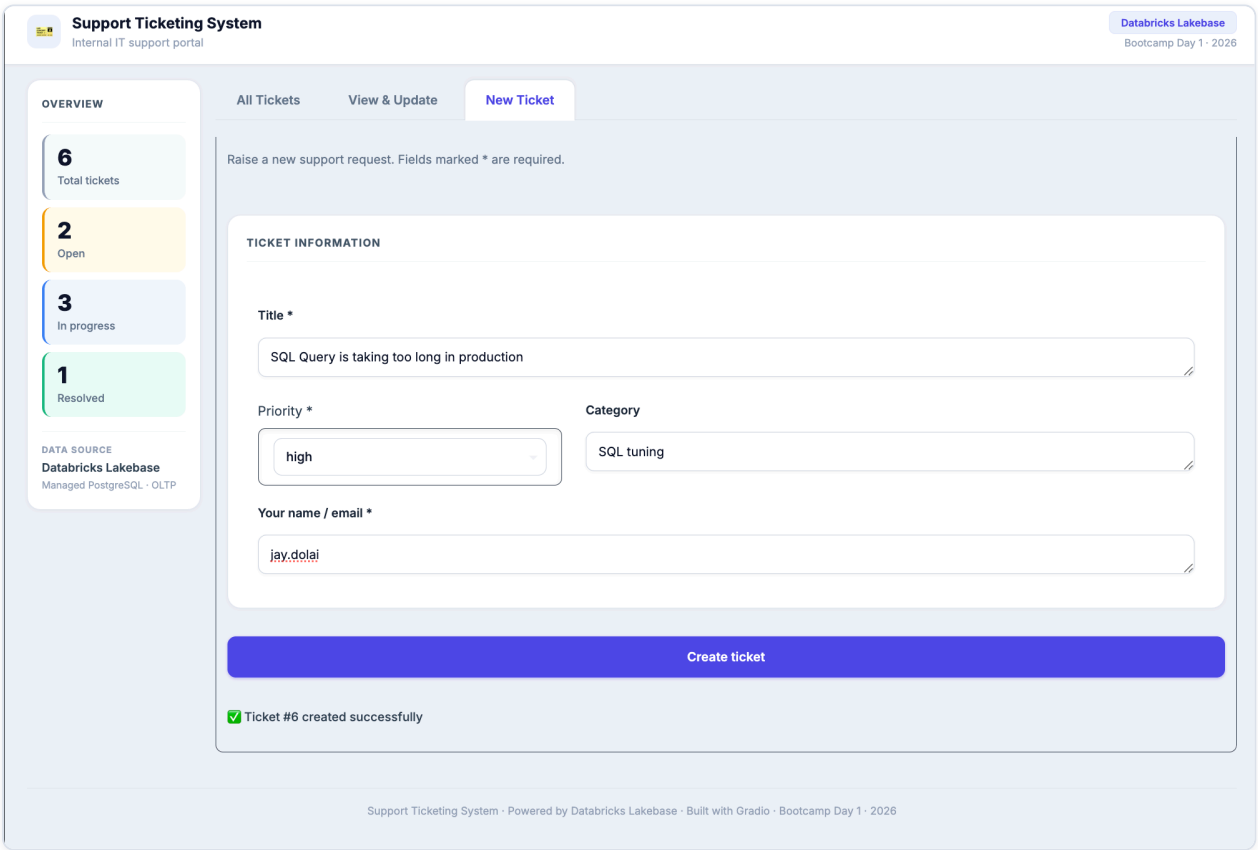


Fig. 6 — New Ticket tab. Validated creation form. Required fields are checked before the insert; on success the app reports the new ticket ID and the sidebar total increments to 6.

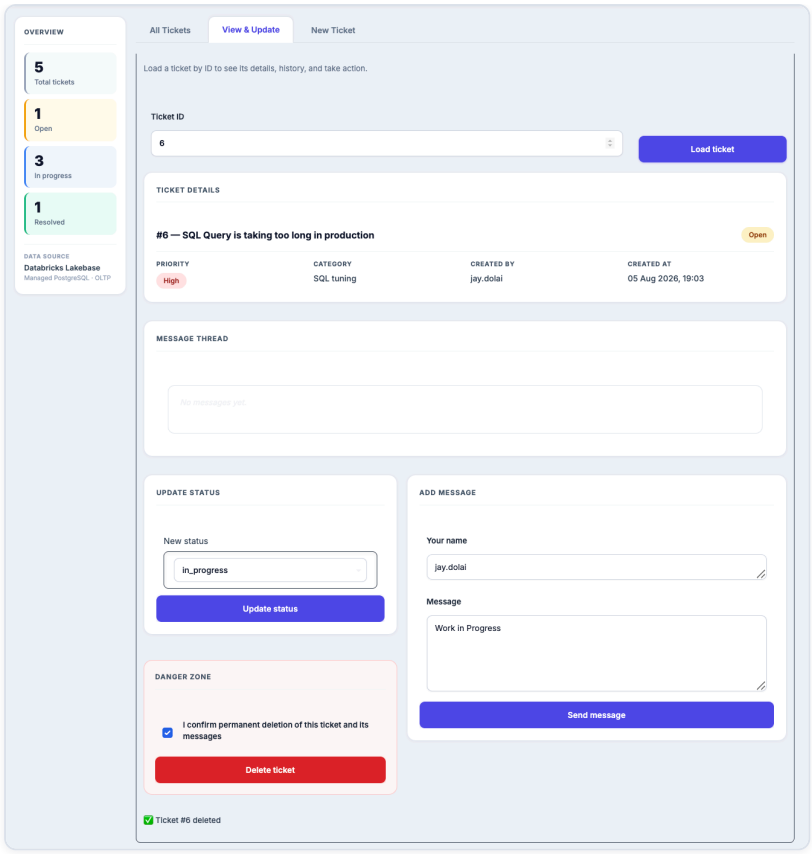


Fig. 7 — Delete with confirmation (bonus). The delete action is a no-op until the confirmation checkbox is ticked. Once confirmed, the ticket is removed and the foreign key cascades the deletion to its messages.

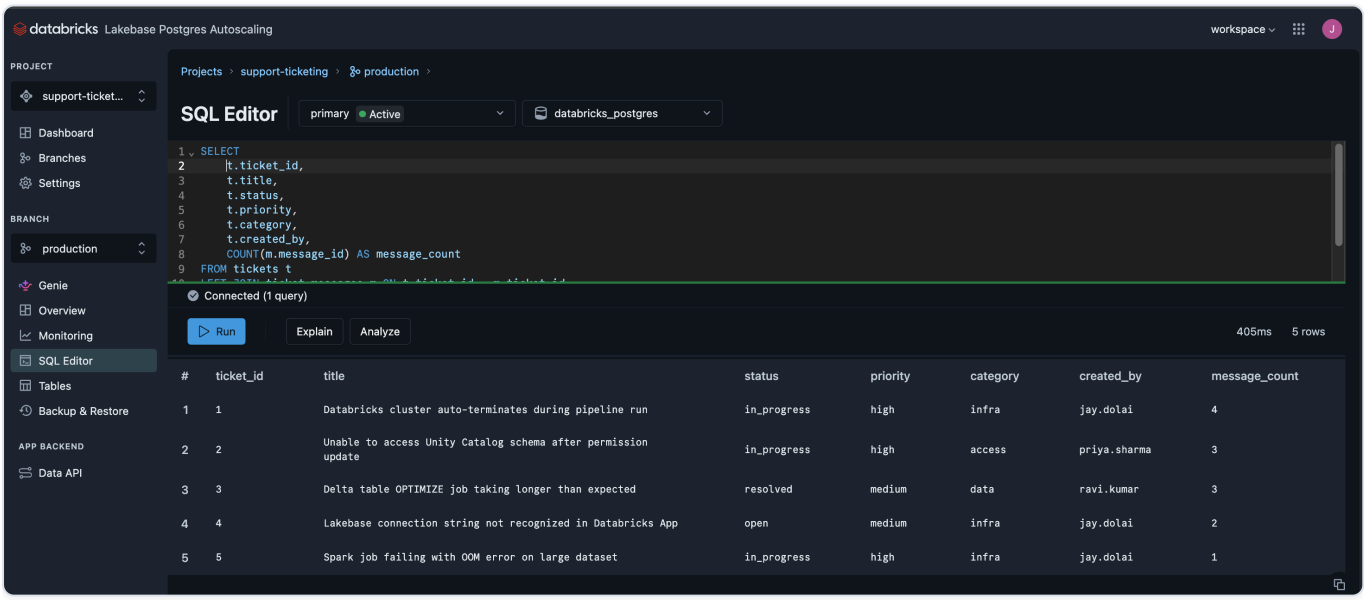


Fig. 8 — Lakebase SQL Editor — verified at the source. The same records queried directly against the `production` branch in the Lakebase SQL Editor: tickets joined to their message counts, returned in 405 ms. Proof the app's writes landed in Postgres.

7 Bonus Features Implemented

Everything below is beyond the required scope of the assignment.

FEATURE	IMPLEMENTATION
Priority & category	Both fields present in the schema, the creation form, the detail card, and the ticket table — rendered as colour-coded chips
Filter by status	Dropdown on the All Tickets tab (All / open / in_progress / resolved), pushed down to a parameterised SQL predicate
Input validation	Required-field checks with inline messages on create, message and status actions — enforced in both the UI handlers and `db.py`
Ticket statistics	Live sidebar dashboard (Total / Open / In Progress / Resolved), refreshed after every mutation
Delete with confirmation	Explicit checkbox required before the delete action is accepted; cascade removes the thread
Improved visual design	Corporate helpdesk-style UI with ~370 lines of custom CSS: status/priority chips, card panels, a danger zone, and a sticky dashboard sidebar

8 Security Posture

No secrets in git	No PAT, connection string, or client secret is committed. `.gitignore` excludes `.env`, `*.pem`, `__pycache__/`, `.DS_Store` and `.streamlit/secrets.toml`.
No static tokens	The app authenticates as its own service principal and mints a short-lived JWT per connection — nothing long-lived to leak or rotate.
Transport	Every connection uses `sslmode=require`; credentials are URL-encoded before being placed in the connection string.
SQL injection	All queries are parameterised with `%s` placeholders — ticket titles and message bodies are never interpolated into SQL.
Input allow-listing	`update_status()` rejects any value outside `{open, in_progress, resolved}` before reaching the database.
Failure handling	Every write is wrapped in `try / except / rollback / finally close`, so a failed statement cannot leave a connection or transaction open.

9 Running It Locally

The app authenticates as a Databricks service principal, so a local run needs the same three variables Databricks Apps injects in production. Create a service principal with OAuth credentials in the workspace and grant it a Lakebase role first.

```
git clone https://github.com/demonjd2026-afk/lakebase-support-ticketing.git
cd lakebase-support-ticketing

python -m venv .venv && source .venv/bin/activate
pip install -r app/requirements.txt

export DATABRICKS_HOST="dbc-291b687e-da89.cloud.databricks.com"
export DATABRICKS_CLIENT_ID="<service-principal-application-id>"
export DATABRICKS_CLIENT_SECRET="<service-principal-oauth-secret>"

# app.py imports db.py as a sibling module, so run from inside app/
cd app && python app.py            # serves on http://localhost:8000

# connection check without launching the UI
cd app && python -c "import db; print(db.get_stats())"
```

The Lakebase host and database name are module-level constants at the top of `app/db.py` — point them at a different Lakebase instance if needed.

10 Reflection

What was the most difficult part?

Getting the Databricks App to authenticate against Lakebase. Lakebase's OAuth connections require a JWT access token — not a personal access token — so several intermediate approaches (raw PAT, hardcoded connection strings, mismatched hostnames) each failed with a different, initially confusing error. The working path was to have the app's own service principal exchange its injected `DATABRICKS_CLIENT_ID` / `DATABRICKS_CLIENT_SECRET` for a JWT via the OIDC `client_credentials` grant, and to grant that principal a Lakebase role on the branch. Notably, ticket #4 in the seed data documents this exact failure — the debugging trail became part of the demo data.

How is Lakebase different from a traditional analytics table?

Lakebase is managed PostgreSQL exposed over the standard wire protocol, giving row-level ACID transactions and millisecond read/write latency — exactly what an interactive app doing single-row inserts and updates needs. A Delta table is optimised for large, append-oriented batch writes and file-level commits; using it for a live app's create / update / delete traffic would be both slow and awkward compared with Lakebase's transactional model. The practical difference shows up in the delete path: a foreign key with `ON DELETE CASCADE` removes a ticket and its entire thread atomically, which has no clean equivalent in Delta.

What feature would you add next?

An AI agent layer on top of the same Lakebase tables — using Databricks Model Serving to auto-triage new tickets (suggesting priority and category from the title), draft reply messages for the support queue, and summarise long threads on load. Because the operational data already lives in Postgres, the agent can read and write it transactionally rather than through a batch pipeline.

Submission checklist

- ✓ Databricks App URL — https://support-ticketing-7474654640109575.aws.databricksapps.com
- ✓ Source code — github.com/demonjd2026-afk/lakebase-support-ticketing
- ✓ Screenshots of the deployed application — Figs. 1–7
- ✓ Screenshot of the Lakebase tables and sample records — Fig. 8
- ✓ Schema DDL and seed data — `sql/01_create_schema.sql`, `sql/02_seed_data.sql`
- ✓ Reflection — Section 10